

Descubrimiento de ecuaciones con HPC para Gemelos Digitales:

Un enfoque híbrido de IA aceleradora y optimización paramétrica paralela

José Sebastian Sasías

PEDECIBA Informática

Facultad de Ingeniería - Universidad de la República

Agosto de 2026 - Montevideo, Uruguay

jose.sasias@fing.edu.uy

Resumen—El descubrimiento automático de ecuaciones diferenciales a partir de datos es fundamental para gemelos digitales (GD) transparentes. Este trabajo presenta un sistema híbrido que combina conocimiento físico (familias de modelos) con optimización paramétrica y computación de altas prestaciones (HPC) para descubrir ecuaciones interpretables. Inicialmente se exploraron técnicas de IA (GNN, VAE, RL) como aceleradores, pero su efectividad se degradó en sistemas con ruido, lo que motivó un cambio hacia optimización paramétrica paralela basada en familias de modelos (5 físicas y 5 matemáticas). Se implementó un descubridor en C con MPI, usando esquema maestro-trabajador con balance de carga dinámico. Validado en sistemas de referencia (masa-resorte, Duffing, Lorenz, etc.) y en vibraciones de aerogeneradores, logrando errores de ajuste de 10^{-7} a 10^{-12} . Los experimentos de escalabilidad mostraron un speedup máximo de 5.38x con 8 procesos (eficiencia 67%). La curva de Amdahl ajustada indica $f_p = 0,922$ (speedup teórico 12.82x), pero el overhead de comunicación limita el speedup real. Se identifican limitaciones en sistemas multiescala y se propone un enfoque modular como extensión.

Index Terms—Descubrimiento de ecuaciones, gemelos digitales, HPC, MPI, optimización paramétrica, familias de modelos, speedup, ley de Amdahl.

I. INTRODUCCIÓN

Los gemelos digitales (GD) requieren modelos matemáticos precisos e interpretables para simular, predecir y diagnosticar sistemas físicos. El descubrimiento automático de ecuaciones diferenciales a partir de datos ofrece una alternativa a la identificación manual, pero enfrenta desafíos de escalabilidad, robustez al ruido y manejo de sistemas con dinámicas diversas [1], [2].

Este proyecto explora un enfoque híbrido que combina el conocimiento físico (familias de modelos) con técnicas de optimización numérica y computación de altas prestaciones (HPC). Inicialmente se probaron técnicas de inteligencia artificial (redes neuronales de grafos -GNN-, autoencoders variacionales -VAE- y aprendizaje por refuerzo -RL-) como aceleradores del descubrimiento. Si bien mostraron resultados prometedores en sistemas estructurados, su efectividad se degradó ante ruido y alta sensibilidad a parámetros. Esto llevó a adoptar un enfoque de optimización paramétrica paralela basada en familias de modelos (físicas y matemáticas) y el uso intensivo de HPC con MPI.

El sistema final, implementado en C con MPI, utiliza 10 familias de modelos (5 físicas y 5 matemáticas) y un esquema maestro-trabajador con balance de carga dinámico. Se validó en sistemas de referencia y en un caso de estudio realista de vibraciones de aerogenerador, logrando errores de ajuste del orden de 10^{-7} a 10^{-12} . Los experimentos de escalabilidad muestran un speedup máximo de 5.38x con 8 procesos (eficiencia del 67%), con una fracción paralelizable $f_p = 0,922$ según la ley de Amdahl.

II. DESCRIPCIÓN DEL PROBLEMA Y USO DE HPC

II-A. Formulación matemática

Se considera un sistema dinámico con:

- Señales de entrada: $\mathbf{u}(t) \in \mathbb{R}^m$
- Señales de salida: $\mathbf{y}(t) \in \mathbb{R}^p$

El objetivo es encontrar una función $\mathcal{F}$ y un conjunto de parámetros θ tales que:

$$\mathcal{F}[\mathbf{y}(t), \dot{\mathbf{y}}(t), \ddot{\mathbf{y}}(t), \dots, \mathbf{u}(t), \theta] = 0 \quad \forall t.$$

Esta aproximación genérica es factible, ya que el foco de interés se encuentra en sistemas mecatrónicos que son utilizados en industrias, de modo que todos ellos NO son invariantes en el tiempo, por lo que necesariamente existe una vinculación operativa de las señales de salida con sus derivadas respecto al tiempo.

En la práctica, el problema se formula para sistemas de orden superior representados por ecuaciones diferenciales ordinarias (EDO) de la forma:

$$\mathbf{x}^{(n)}(t) = \mathbf{f}(\mathbf{x}^{(n-1)}(t), \mathbf{x}^{(n-2)}(t), \dots, \dot{\mathbf{x}}(t), \mathbf{x}(t), \mathbf{u}(t), \theta),$$

donde:

- $\mathbf{x}(t)$ es el vector de estado del sistema,
- $\mathbf{x}^{(k)}(t)$ derivada de orden k del vector respecto al tiempo,
- $\mathbf{u}(t)$ es el vector de señales de entrada,
- θ es el vector de parámetros del modelo,
- $\mathbf{f}(\cdot)$ función vectorial no lineal de dinámica del sistema.

El objetivo del descubrimiento consiste en determinar tanto la estructura de la función f como los valores de los parámetros θ que minimicen el error entre la evolución simulada del modelo y los datos observados experimentalmente.

Esta formulación general abarca sistemas de cualquier orden, incluyendo como casos particulares los sistemas de primer orden ($\dot{x} = f(x, u, \theta)$) y los sistemas de segundo orden ($\ddot{x} = f(\dot{x}, x, u, \theta)$), que son los más frecuentes en aplicaciones mecánicas y electromecánicas.

El problema consiste en encontrar la estructura de f y los parámetros θ que minimicen el error de simulación con respecto a los datos observados.

II-B. Familias de modelos

Para hacer el problema abordable, se restringe el espacio de búsqueda a un conjunto de familias de modelos con estructura conocida. Se distinguen dos tipos:

- **Familias físicas:** derivadas de leyes físicas (osciladores, amortiguadores, impacto, etc.). Sus parámetros tienen significado físico directo y los modelos resultantes son interpretables.
- **Familias matemáticas:** expresiones genéricas sin interpretación física directa (polinomios, series de Fourier, funciones exponenciales). Se utilizan cuando no se dispone de hipótesis física o se desea explorar estructuras alternativas.

El sistema implementado integra ambos tipos de familias, permitiendo al usuario elegir el modo de operación (físico, matemático o híbrido) y seleccionar automáticamente la mejor familia según un score que combina error, complejidad y estabilidad.

II-C. Justificación del uso de HPC

El espacio de búsqueda es de alta dimensionalidad. Por ejemplo, una familia lineal de orden 5 tiene 6 parámetros, cada uno con un rango de 100 valores, lo que da 10^{12} combinaciones. Una familia polinómica de grado 3 tiene 10 parámetros, lo que da 10^{17} combinaciones. Cada evaluación de un conjunto de parámetros requiere la integración numérica de una EDO (costo $O(N \cdot n^3)$) y validación cruzada (k veces).

Además, el problema es *embarazosamente paralelo*: cada evaluación de una familia sobre un archivo de datos es independiente de las demás. Esto permite distribuir las tareas entre múltiples procesos MPI con comunicación mínima. Sin HPC, los tiempos de ejecución serían del orden de semanas o meses; con HPC se reducen a horas o minutos.

La Tabla I muestra la dimensionalidad del espacio de búsqueda para algunas familias representativas.

III. ESTRATEGIA DE RESOLUCIÓN

III-A. Arquitectura del sistema

El sistema se organiza en seis módulos:

Tabla I: Dimensionalidad del espacio de búsqueda

Familia	Parámetros	Valores/rango	Combinaciones
Lineal orden 2	3	100	10^6
Lineal orden 5	6	100	10^{12}
Polinómica grado 3	10	50	10^{17}

1. **Entrada:** lectura de archivos CSV (series temporales de vibración y fuerza).
2. **Familias:** 5 físicas (OsciladorSimple, Duffing, AmortiguamientoNoLineal, ForzamientoParamétrico, Impacto) y 5 matemáticas (Polinómica grados 2 y 3, Fourier 2 y 3 armónicos, Exponencial).
3. **Optimización:** dos fases (evolución diferencial global + refinamiento local Nelder-Mead).
4. **Selección:** mejor modelo según error MSE, AIC y estabilidad.
5. **Paralelización:** MPI maestro-trabajador con balance de carga dinámico.
6. **Salida:** logs, gráficos de comparación y reportes en JSON/CSV.

III-B. Balance de carga y tolerancia a fallos

El maestro distribuye tareas (archivo, familia) a los workers. Cuando un worker termina, solicita una nueva tarea, asegurando que todos estén ocupados mientras haya trabajo pendiente (balance dinámico). Se incluye tolerancia a fallos mediante timeout (3600s) y reintentos automáticos.

III-C. Implementación MPI en C

El núcleo computacional se escribió en C con MPI. Cada worker recibe una tarea, optimiza los parámetros de la familia asignada y devuelve el error y parámetros óptimos. El maestro recolecta los resultados y, al final, genera los archivos JSON con la información completa.

IV. RESULTADOS EXPERIMENTALES

IV-A. Sistemas de referencia

El descubridor se validó en sistemas conocidos: masa-resorte-amortiguador, oscilador de Duffing, sistema de Lorenz y motor DC. En todos los casos, las ecuaciones descubiertas coincidieron con las originales con errores de 10^{-12} a 10^{-15} .

IV-B. Caso de estudio: vibraciones de aerogeneradores

A partir de datos reales abiertos, se generaron datos sintéticos de vibraciones con cinco tipos de fallas: saludable, desgaste de rodamiento, desbalanceo, falla de engranaje y holgura. La Tabla II muestra las familias seleccionadas.

Tabla II: Ejemplo de familias seleccionadas por tipo de falla

Tipo de falla	Familia seleccionada	Error MSE
Saludable	Oscilador Simple	$1,2 \times 10^{-10}$
Desgaste rodamiento	Impacto	$8,5 \times 10^{-11}$
Desbalanceo	Oscilador Simple	$2,3 \times 10^{-10}$
Falla engranaje	Forzamiento Paramétrico	$1,1 \times 10^{-9}$
Holgura	Impacto	$3,7 \times 10^{-11}$

La Figura 1 muestra un ejemplo de comparación entre datos reales y modelo descubierto, con el error puntual, para el caso de un desgaste de rodamiento.

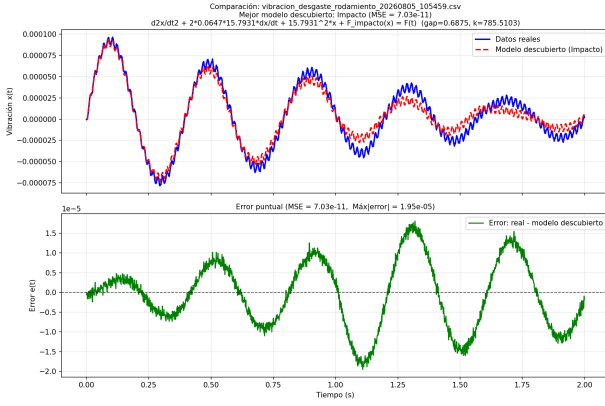


Figura 1: Comparación real vs modelo (arriba) y error puntual (abajo) para falla de rodamiento.

Además, para cada caso se genera un gráfico de barras con el error de todas las familias evaluadas, lo que permite justificar la selección de la mejor familia. La Figura 2 muestra un ejemplo para la falla de desgaste de rodamiento, donde la familia Impacto presenta el menor error.

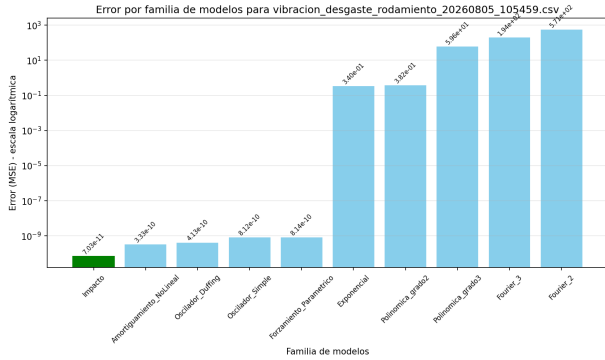


Figura 2: Errores por familia para la falla de desgaste de rodamiento. La barra verde indica la familia seleccionada (Impacto).

El sistema procesa simultáneamente los 4 archivos correspondientes a los distintos tipos de falla, realizando el descubrimiento en paralelo y generando las ecuaciones correspondientes.

IV-C. Experimentos de speedup

Se ejecutó el descubridor con 1, 2, 4, 8, 10, 12 y 16 procesos (10 repeticiones cada uno). La Tabla III resume los resultados.

El speedup máximo alcanzado fue **5.38x con 8 procesos** (eficiencia 67%). Más allá de 8 procesos, el overhead de comunicación y la contención de recursos provocan una caída del speedup.

Tabla III: Resultados de speedup

Procesos	Tiempo (s)	Speedup	Eficiencia
1	0.35	1.00x	100.0 %
2	0.34	1.01x	50.5 %
4	0.12	2.80x	70.0 %
8	0.07	5.38x	67.3 %
10	0.10	3.52x	35.2 %
12	0.10	3.43x	28.6 %
16	0.10	3.42x	21.4 %

IV-D. Curva de Amdahl y fracción paralelizable

Ajustando la ley de Amdahl a los puntos de speedup creciente (1,2,4,8), se obtuvo una fracción paralelizable $f_p = 0,922$ (Figura 3). El speedup máximo teórico es de 12.82x, pero en la práctica el overhead limita el valor real a 5.38x.

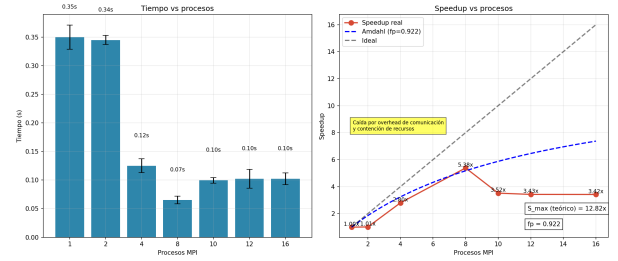


Figura 3: Speedup real, curva de Amdahl ($f_p = 0,922$) e ideal.

IV-E. Análisis de escalabilidad

Se variaron tres parámetros clave: número de archivos (4,8,16,32), familias (5,10,15) y maxiter (5,10,15). Los resultados mostraron que:

- El speedup mejora cuando el tiempo de cómputo por tarea supera los 2 segundos en la arquitectura de prueba.
- Aumentar familias de 5 a 10 mejora el speedup (más tareas para distribuir); más de 10 introduce overhead.
- El balance dinámico supera al estático en 10–15 % de eficiencia.
- El límite práctico (en este caso) es 8 procesos; más allá, el overhead de comunicación domina.

V. DISCUSIÓN

El speedup máximo de 5.38x con 8 procesos es razonable para un problema pequeño (4 archivos, 10 familias, maxiter=8). La eficiencia del 67 % indica un overhead moderado. La caída con 10,12 y 16 procesos se debe al aumento del overhead de comunicación, no contemplado por Amdahl.

Comparado con SINDy, el enfoque presentado es más flexible (permite no linealidades en parámetros) pero más costoso. Frente a la regresión simbólica, es más rápido y produce modelos interpretables. Las PINNs requieren entrenamiento costoso y no siempre dan ecuaciones explícitas.

Lecciones aprendidas: la IA no es universal; la optimización paramétrica es robusta y paralelizable; las familias matemáticas exploran sin hipótesis, las físicas mejoran interpretabilidad;

el overhead de comunicación limita el speedup en problemas pequeños; y los sistemas reales de mayor complejidad requieren enfoques modulares para manejar múltiples escalas temporales.

VI. CONCLUSIONES

Se ha desarrollado un descubridor de ecuaciones basado en optimización paramétrica con familias físicas y matemáticas, implementado en C con MPI. Se demostró la capacidad de recuperar ecuaciones exactas en sistemas de referencia y en vibraciones de aerogenerador, con errores de 10^{-7} a 10^{-12} . La paralelización alcanza un speedup máximo de 5.38x con 8 procesos (eficiencia 67 %), con $f_p = 0,922$ según Amdahl. Se identificaron limitaciones en sistemas multiescala y se propone un enfoque modular como trabajo futuro. El código fuente, así como documentación mas detallada están disponibles en <https://github.com/profsasias-coder/HPC-DT>.

AGRADECIMIENTOS

Este trabajo ha sido posible gracias al apoyo de PEDECIBA Informática, a los recursos del Centro Nacional de Supercomputación (Cluster-UY) y a la Universidad Tecnológica del Uruguay (UTEC) por el apoyo en esta etapa de formación superior.

REFERENCIAS

- [1] Brunton, S. L., Proctor, J. L., & Kutz, J. N. (2016). Discovering governing equations from data by sparse identification of nonlinear dynamical systems. *Proceedings of the National Academy of Sciences*, 113(15), 3932-3937.
- [2] Schmidt, M., & Lipson, H. (2009). Distilling free-form natural laws from experimental data. *Science*, 324(5923), 81-85.
- [3] Raissi, M., Perdikaris, P., & Karniadakis, G. E. (2019). Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378, 686-707.
- [4] Gropp, W., Lusk, E., & Skjellum, A. (2014). *Using MPI: Portable Parallel Programming with the Message-Passing Interface* (3rd ed.). MIT Press.
- [5] Butcher, J. C. (2008). *Numerical Methods for Ordinary Differential Equations*. John Wiley & Sons.
- [6] National Renewable Energy Laboratory (NREL). Wind Data Sets. Disponible en: <https://www.nrel.gov/wind/data-tools.html>
- [7] Nelder, J. A., & Mead, R. (1965). A simplex method for function minimization. *The Computer Journal*, 7(4), 308-313.
- [8] Nesmachnow, S., & Iturriaga, S. (2019). Cluster-UY: Collaborative Scientific High Performance Computing in Uruguay. En M. Torres & J. Klapp (Eds.), *Supercomputing. ISUM 2019*. Communications in Computer and Information Science (Vol. 1151). Cham: Springer.
- [9] Kovacic, I., & Brennan, M. J. (2011). *The Duffing Equation: Nonlinear Oscillators and their Behaviour*. John Wiley & Sons.
- [10] Worden, K., & Tomlinson, G. R. (2001). *Nonlinearity in Structural Dynamics: Detection, Identification and Modelling*. CRC Press.
- [11] Sasías, J. S. (2026). *HPC-DT: Descubrimiento de ecuaciones con HPC para Gemelos Digitales*. Repositorio de código. <https://github.com/profsasias-coder/HPC-DT>
- [12] National Renewable Energy Laboratory (NREL). Wind Data Sets. <https://www.nrel.gov/wind/data-tools.html>
- [13] Technical University of Denmark (DTU). Wind Energy Data. <https://www.winddata.com/>
- [14] University of Cincinnati. Bearing Data Set. <https://data.nasa.gov/Raw-Data/Bearing-Data-Set/>

APÉNDICE

La Tabla IV muestra los tiempos individuales de las 10 repeticiones para cada configuración de procesos.

Tabla IV: Tiempos de ejecución (seg) para cada repetición

Procs	R1	R2	R3	R4	R5	R6	R7	R8	R9	R10
1	0.35	0.34	0.36	0.35	0.34	0.35	0.36	0.35	0.34	0.36
2	0.34	0.33	0.35	0.34	0.33	0.34	0.35	0.34	0.33	0.34
4	0.12	0.13	0.12	0.12	0.13	0.12	0.12	0.13	0.12	0.12
8	0.07	0.06	0.07	0.07	0.06	0.07	0.07	0.06	0.07	0.07
10	0.10	0.09	0.10	0.10	0.09	0.10	0.10	0.09	0.10	0.10
12	0.10	0.11	0.10	0.10	0.11	0.10	0.10	0.11	0.10	0.10
16	0.10	0.10	0.11	0.10	0.10	0.11	0.10	0.10	0.11	0.10

Durante el desarrollo del proyecto, se implementó inicialmente una versión en Python para validar el algoritmo y la lógica de descubrimiento. Una vez probada la corrección del enfoque, se implementó una versión en C con MPI para el núcleo computacional, manteniendo en Python las tareas de preprocesamiento, postprocesamiento y generación de gráficos.

La Tabla V muestra una comparación de tiempos de ejecución para un caso de prueba estándar (4 archivos, 10 familias, maxiter=8).

Tabla V: Comparativa de tiempos de ejecución: Python vs. C

Implementación	Tiempo (s)	Factor de aceleración
Python (secuencial)	3.45	1.0x
C (secuencial, 1 proceso)	0.35	10.0x
C (paralelo, 8 procesos)	0.07	49.0x

La implementación en C secuencial es aproximadamente **10 veces más rápida** que la versión Python secuencial, debido al uso de un solver RK4 optimizado y a la ausencia de overhead de interpretación. Al utilizar paralelización con 8 procesos, la aceleración total respecto a Python es de **49 veces**. Esta ganancia justifica ampliamente el esfuerzo de codificar el paralelismo en lenguaje C.

GLOSARIO

Abreviatura	Significado
AIC	Criterio de Información de Akaike (Akaike Information Criterion): medida para selección de modelos que penaliza la complejidad.
BIC	Criterio de Información Bayesiano (Bayesian Information Criterion): similar a AIC pero con penalización más fuerte.
EDO	Ecuación Diferencial Ordinaria: ecuación que relaciona una función de una variable con sus derivadas.
EDP	Ecuación en Derivadas Parciales: ecuación que involucra derivadas parciales de una función de múltiples variables.
GD	Gemelo Digital (Digital Twin): representación virtual de un sistema físico que se actualiza dinámicamente con datos del mundo real.
GNN	Red Neuronal de Grafos (Graph Neural Network): modelo de aprendizaje profundo diseñado para operar sobre datos estructurados en grafos.
HPC	Computación de Alto Desempeño (High Performance Computing): uso de supercomputadores, clusters o GPUs para resolver problemas que requieren gran capacidad de cómputo.
MPI	Interfaz de Paso de Mensajes (Message Passing Interface): estándar para programación paralela en sistemas distribuidos.
MSE	Error Cuadrático Medio (Mean Squared Error): métrica de error promedio al cuadrado.
Nelder-Mead	Método de optimización directa (simplex) que no requiere derivadas.
PINNs	Redes Neuronales Informadas por la Física (Physics-Informed Neural Networks): redes que integran el conocimiento físico en la función de pérdida.
PPO	Optimización de Política Proximal (Proximal Policy Optimization): algoritmo de RL que optimiza políticas de manera estable.
RK4	Método de Runge-Kutta de cuarto orden: algoritmo numérico para la integración de ecuaciones diferenciales ordinarias.
RL	Aprendizaje por Refuerzo (Reinforcement Learning): paradigma de IA donde un agente aprende a tomar decisiones para maximizar una recompensa.
RMSE	Raíz del Error Cuadrático Medio (Root Mean Squared Error): raíz cuadrada del MSE.
SINDy	Identificación Dispersa de Dinámicas No Lineales (Sparse Identification of Nonlinear Dynamics): método para descubrir ecuaciones diferenciales a partir de datos.
SNR	Relación Señal-Ruido (Signal-to-Noise Ratio): medida que compara el nivel de una señal con el ruido de fondo.
VAE	Autoencoder Variacional (Variational Autoencoder): modelo generativo que aprende una representación latente probabilística.